

Taller 4: gestores de estado - persistencia local

Autor

Daniela Erazo Marin

Docente

David Steven Duran Vallejo

Unidad Central Del Valle

Facultad De Ingeniería

Ingeniería De Sistemas

Tuluá – Valle Del Cauca

2025

Repo: https://github.com/daniela-erazo-marin/curso_flutter

1 Objetivo

Evaluar las habilidades del desarrollador en el uso de **Flutter** para construir una aplicación móvil moderna con:

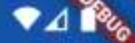
- Gestión de estado y arquitectura limpia.
- Integración con **API REST**.
- **Persistencia local** usando SQLite (sqflite) y modo offline con sincronización.
- Buenas prácticas de desarrollo (documentación, control de versiones).

2 Aplicación: To-Do List

Desarrollar una aplicación Flutter que permita:

- Crear, editar, marcar como completadas y eliminar tareas.
- Listar tareas con filtros básicos (todas, pendientes, completadas).

11:45



To-Do Offline



hacer tarea-2

2025-11-17 23:19:58.901656



hacer tesis

2025-11-17 23:17:59.799808



11:46



DEBUG



Editar tarea

Título

hacer tarea 2

Actualizar

11:46



To-Do Offline

- | | | | |
|-------------------------------------|---|--|---|
| ☒ | Ir al gym
2025-11-17 23:46:45.478984 |  |  |
| ☒ | hacer tarea 2
2025-11-17 23:46:31.254581 |  |  |
| ☐ | hacer tesis
2025-11-17 23:17:59.799808 |  |  |



3 Integración con API

Se crea una FastAPI con Python cumpliendo con el contrato de endpoints.

Endpoints esperados

GET /tasks

POST /tasks

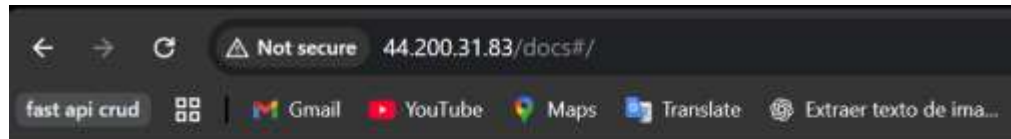
GET /tasks/{id}

PUT /tasks/{id}

DELETE /tasks/{id}

Formato: JSON

Campos mínimos: id, title, completed (bool), updatedAt (ISO8601).



FastAPI 0.1.0 OAS 3.1
/openapi.json

default

GET	/	Read Root
GET	/tasks	Get All Tasks
POST	/tasks	Save Task
GET	/tasks/{task_id}	Get Task
DELETE	/tasks/{task_id}	Delete Task
PUT	/tasks/{task_id}	Update Task

Código de la Fast API:

```
requirements.txt  main.py x
27
28 @app.post("/tasks")  & Daniela E
29 def save_task(task: Task):
30     # Evitar IDs duplicados
31     for existing in tasks:
32         if existing["id"] == task.id:
33             raise HTTPException(status_code=400, detail="ID already exists")
34
35     task.updated_at = datetime.now()
36     tasks.append(task.dict())
37     return task
38
39
40 @app.get("/tasks/{task_id}")  & Daniela E
41 def get_task(task_id: str):
42     for task in tasks:
43         if task["id"] == task_id:
44             return task
45     raise HTTPException(status_code=404, detail="Item not found")
46
47
48 @app.delete("/tasks/{task_id}")  & Daniela E
49 def delete_task(task_id: str):
50     for index, task in enumerate(tasks):
51         if task["id"] == task_id:
52             tasks.pop(index)
53             return {"message": "Task deleted successfully"}
54     raise HTTPException(status_code=404, detail="Item not found")
55
56
57 @app.put("/tasks/{task_id}")  & Daniela E
58 def update_task(task_id: str, updatedTask: Task):
```

Comandos usados para crear la FastApi-Task

```
Terminal Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task>
(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task> pip install fastapi
Collecting fastapi
  Using cached fastapi-0.121.2-py3-none-any.whl.metadata (28 kB)
Collecting starlette<0.50.0,>=0.40.0 (from fastapi)
  Using cached starlette-0.49.3-py3-none-any.whl.metadata (6.4 kB)
Collecting pydantic!=1.8,!1.8.1,!2.0.0,!2.0.1,!2.1.0,<3.0.0,>=1.7.4 (from fastapi)
  Using cached pydantic-2.12.4-py3-none-any.whl.metadata (89 kB)
```

```
Terminal Local x + v
https://aka.ms/PSWindows

(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task>
(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task> pip install fastapi
Collecting fastapi
  Using cached fastapi-0.121.2-py3-none-any.whl.metadata (28 kB)
Collecting starlette<0.50.0,>=0.40.0 (from fastapi)
  Using cached starlette-0.49.3-py3-none-any.whl.metadata (6.4 kB)
Collecting pydantic!=1.8,!1.8.1,!2.0.0,!2.0.1,!2.1.0,<3.0.0,>=1.7.4 (from fastapi)
  Using cached pydantic-2.12.4-py3-none-any.whl.metadata (89 kB)
Collecting typing-extensions>=4.8.0 (from fastapi)
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting annotated-doc>=0.0.2 (from fastapi)
  Using cached annotated_doc-0.0.4-py3-none-any.whl.metadata (6.6 kB)

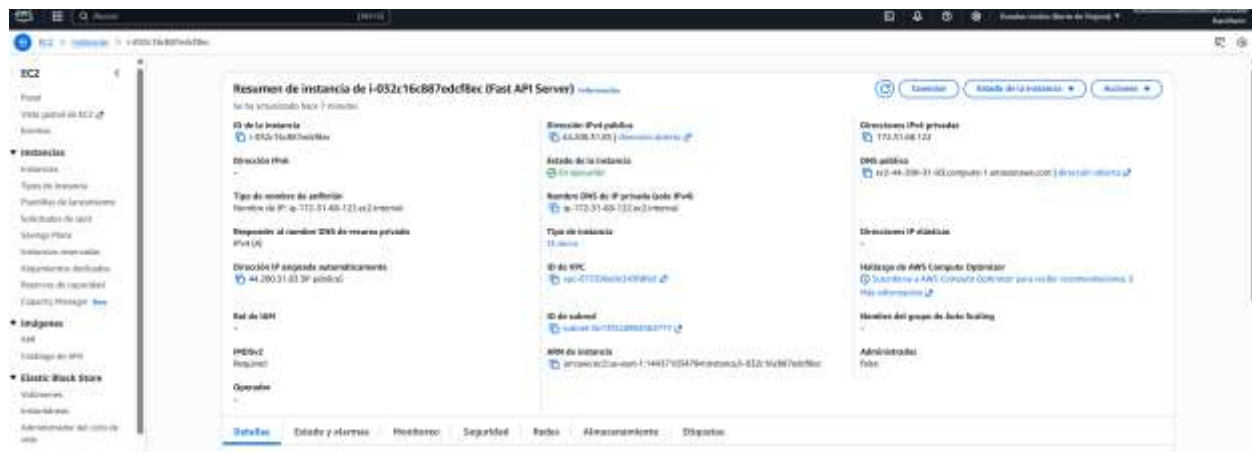
FastApi-Task Python 3.13 (FastApi-Task)
```

```
Terminal Local x + v

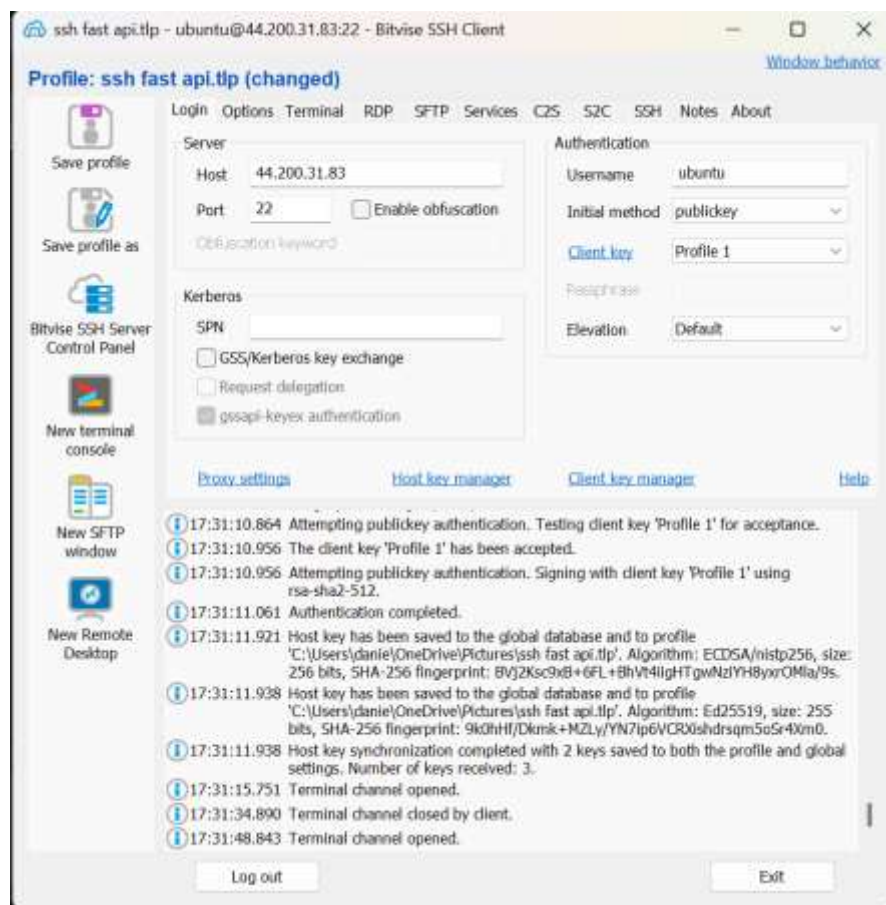
[notice] A new release of pip is available: 25.1.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task>
(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task>
(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task> pip freeze > requeriments.txt
(.venv) PS C:\Users\danie\PycharmProjects\FastApi-Task> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\danie\\PycharmProjects\\FastApi-Task']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [1820] using StatReload
INFO: Started server process [15772]
INFO: Waiting for application startup.
INFO: Application startup complete.

FastApi-Task > main.py 47:15 CRLF UTF-8 4 spaces Python 3.13 (FastApi-Task)
```


[illegible]



Me conecte a la maquina Ubuntu por medio de ssh



Comandos usados:

```
Last login: Mon Nov 17 00:47:48 2025 from 190.108.78.184
ubuntu@ip-172-31-68-122:~$ history
 1  sudo apt-get update
 2  cls
 3  clc
 4  clear
 5  sudo apt install -y python3-pip nginx
 6  sudo vim /etc/nginx/sites-enable
 7  sudo vim /etc/nginx/sites-enable/fastapi_nginx
 8  history
 9  sudo apt update
10  cd ~/FastApi-Task
11  python3 -m venv venv
12  sudo apt install -y python3.12-venv
13  rm -rf venv
14  python3 -m venv venv
15  source venv/bin/activate
16  ls
17  cat requirements.txt
18  pip install -r requirements.txt
19  clear
20  python3 -m uvicorn main:app
21  cd FastApi-Task/
22  python3 -m uvicorn main:app
23  source venv/bin/activate
24  python3 -m uvicorn main:app
25  pwd
26  ls
27  ls -la
28  ls -la venv/bin
29  realpath .
30  sudo nano /etc/systemd/system/fastapi.service
31  sudo systemctl daemon-reload
32  sudo systemctl enable fastapi
33  sudo systemctl start fastapi
34  sudo systemctl status fastapi
35  history
36  c..
37  c ..
38  history
ubuntu@ip-172-31-68-122:~$
```

Fastapi.service corriendo en la instancia ec2 con daemon

bash

 Copy code

```
sudo nano /etc/systemd/system/fastapi.service
```

ini

 Copy code

```
[Unit]
Description=FastAPI Service
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/FastApi-Task
ExecStart=/home/ubuntu/FastApi-Task/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
Restart=always
RestartSec=3
Environment="PATH=/home/ubuntu/FastApi-Task/venv/bin"

[Install]
WantedBy=multi-user.target
```

bash

 Copy code

```
sudo systemctl daemon-reload
sudo systemctl enable fastapi
sudo systemctl start fastapi
```

bash

 Copy code

```
sudo systemctl status fastapi
```

Chat gpt usado para errores:

<https://chatgpt.com/share/691a7558-e3e4-8001-b7a0-e51e1d88013e>

Video guía para crear instancia en ec2:

<https://www.youtube.com/watch?v=SgSnz7kW-Ko>

Repo guía:

<https://github.com/pixegami/fastapi-tutorial>

Persistencia local (SQLite)

Usar el paquete sqflite para almacenar las tareas localmente.

Esquema de ejemplo

```
CREATE TABLE tasks (  
  id TEXT PRIMARY KEY,  
  title TEXT NOT NULL,  
  completed INTEGER NOT NULL DEFAULT 0,  
  updated_at TEXT NOT NULL,  
  deleted INTEGER NOT NULL DEFAULT 0  
);
```

```
class TaskDB {  
  TaskDB._init();  
  
  Future<Database> get database async {  
    if (_database != null) return _database!  
    _database = await _initDB('tasks.db');  
    return _database!  
  }  
  
  Future<Database> _initDB(String filePath) async {  
    final dbPath = await getDatabasesPath();  
    final path = join(dbPath, filePath);  
    return await openDatabase(path, version: 1, onCreate: _createDB);  
  }  
  
  Future _createDB(Database db, int version) async {  
    await db.execute('''  
      CREATE TABLE tasks (  
        id TEXT PRIMARY KEY,  
        title TEXT NOT NULL,  
        completed INTEGER NOT NULL DEFAULT 0,  
        updated_at TEXT NOT NULL,  
        deleted INTEGER NOT NULL DEFAULT 0  
      );  
    ''');
```

5 Sincronización (cuando vuelve la conexión)

Implementar una tabla de cola de operaciones para sincronizar con el servidor:

```
queue_operations(  
    id TEXT PRIMARY KEY,  
    entity TEXT,  
    entity_id TEXT,  
    op TEXT,      -- CREATE | UPDATE | DELETE  
    payload TEXT,  
    created_at INTEGER,  
    attempt_count INTEGER,  
    last_error TEXT  
);
```

```
await db.execute('''  
    CREATE TABLE queue_operations (  
        id TEXT PRIMARY KEY,  
        entity TEXT,  
        entity_id TEXT,  
        op TEXT,  
        payload TEXT,  
        created_at INTEGER,  
        attempt_count INTEGER,  
        last_error TEXT  
    );  
''')
```


Flujo de sincronización

1. La app detecta conectividad (connectivity_plus) o al abrirse.
2. Toma operaciones pendientes y las envía al backend.
3. Si la API responde con éxito, marca la operación como completada y actualiza syncedAt.
4. Si falla, reintenta con backoff exponencial y registra last_error.

```
15 class SyncService {
26 // -----
27 // AUTO-SYNC (cada X segundos si hay internet)
28 // -----
29 void startAutoSync({int intervalSeconds = 8}) {
30   _timer?.cancel();
31   _timer = Timer.periodic(Duration(seconds: intervalSeconds), (_) async {
32     final conn = await Connectivity().checkConnectivity();
33     if (conn != ConnectivityResult.none) { The type of the right operand ('ConnectivityResult'
34       await syncQueue();
35     }
36   }); // Timer.periodic
37 }
38
39 void stop() {
40   _timer?.cancel();
41   _running = false;
42 }
43
44 // -----
45 // PROCESAR COLA
46 // -----
47 Future<void> syncQueue() async {
48   if (_running) return;
49   _running = true;
50
51   final queue = await local.getQueue();
52
53   for (final op in queue) {
54     final opId = op['id'] as String;
55     final opType = op['op'] as String;
56     final entityId = op['entity_id'] as String;
57     final payload = op['payload'] as String;
58     int attempt = (op['attempt_count'] ?? 0) as int;
59
60     _logger.i("Processing queue item: $opId type:$opType attempt:$attempt");
61
62     TaskModel? model;
```

Comunicación con la API:

The screenshot shows a web browser window on the left and a mobile app interface on the right. The browser window displays a REST client interface with the following details:

- Request URL:** `http://44.200.31.83/tasks`
- Server response:** `200`
- Response body:**

```
{
  "id": "5f527480-78e5-4944-85c3-8fa230113154",
  "title": "ir al gym",
  "completed": true,
  "updated_at": "2025-11-17 23:46:57.915210"
},
{
  "id": "a6b11a65-ba6c-4a4a-8845-899170800421",
  "title": "holaaaa",
  "completed": false,
  "updated_at": "2025-11-18 00:02:55.191325"
}
```
- Response headers:**

```
connection: keep-alive
content-length: 258
content-type: application/json
date: Tue, 18 Nov 2025 00:02:58 GMT
server: nginx/1.24.0 (Ubuntu)
```
- Responses table:**

Code	Description	Links
200	Successful Response	No links

The mobile app interface on the right shows a list of tasks under the heading "To-Do Offline". The tasks are:

- ☐ holaaaa (2025-11-18 00:02:55.191325)
- ☒ ir al gym (2025-11-17 23:46:57.915210)

The screenshot shows a web browser window on the left and a mobile app interface on the right. The browser window displays a REST client interface with the following details:

- Request URL:** `http://44.200.31.83/tasks`
- Server response:** `200`
- Response body:**

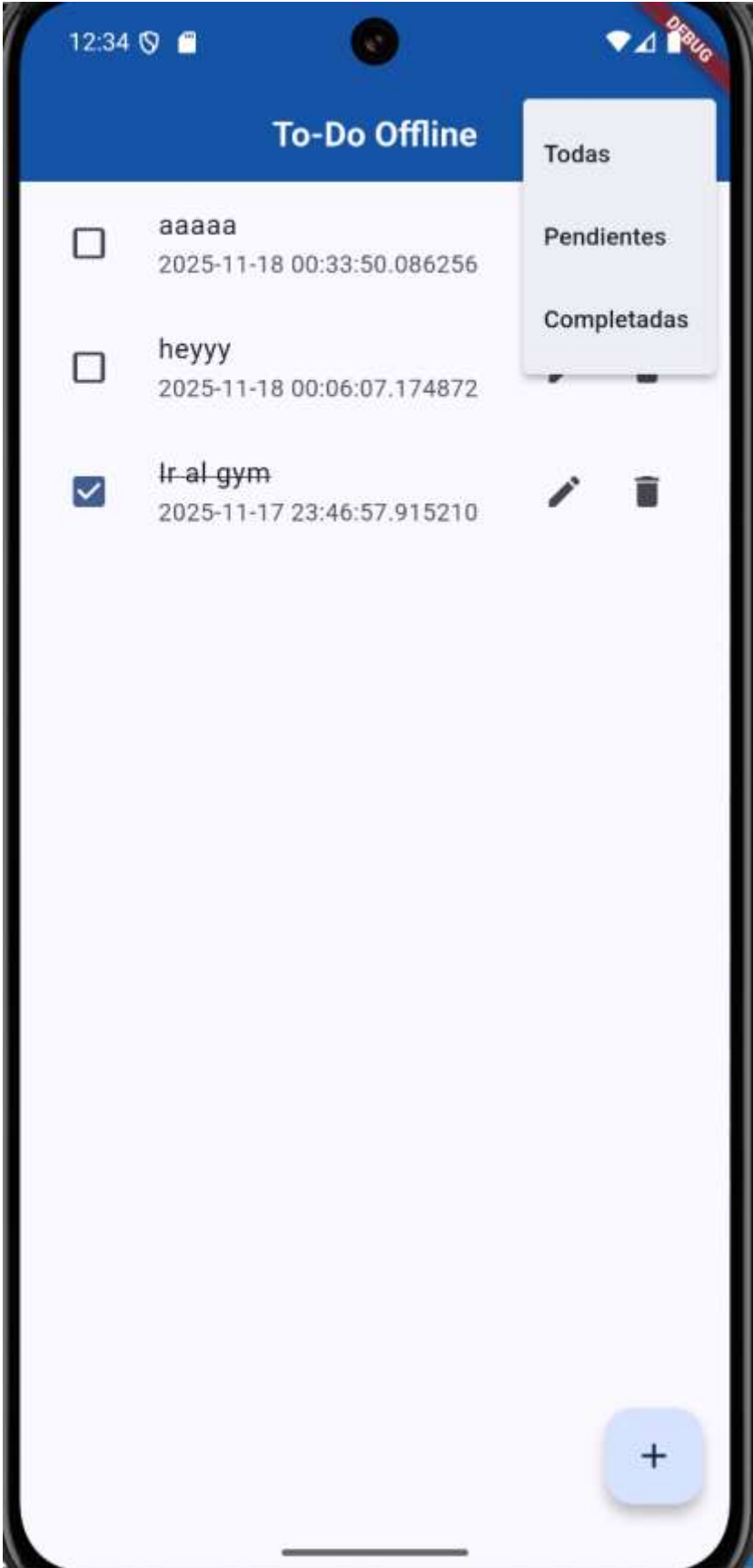
```
{
  "id": "5f527480-78e5-4944-85c3-8fa230113154",
  "title": "ir al gym",
  "completed": true,
  "updated_at": "2025-11-17 23:46:57.915210"
},
{
  "id": "a6b11a65-ba6c-4a4a-8845-899170800421",
  "title": "holaaaa",
  "completed": false,
  "updated_at": "2025-11-18 00:02:55.191325"
},
{
  "id": "d15b0680-3585-4a3b-8a3a-f4f4a5338073",
  "title": "aaaaaa",
  "completed": true,
  "updated_at": "2025-11-18 00:03:59.186603"
}
```
- Response headers:**

```
connection: keep-alive
content-length: 372
content-type: application/json
date: Tue, 18 Nov 2025 00:04:01 GMT
server: nginx/1.24.0 (Ubuntu)
```
- Responses table:**

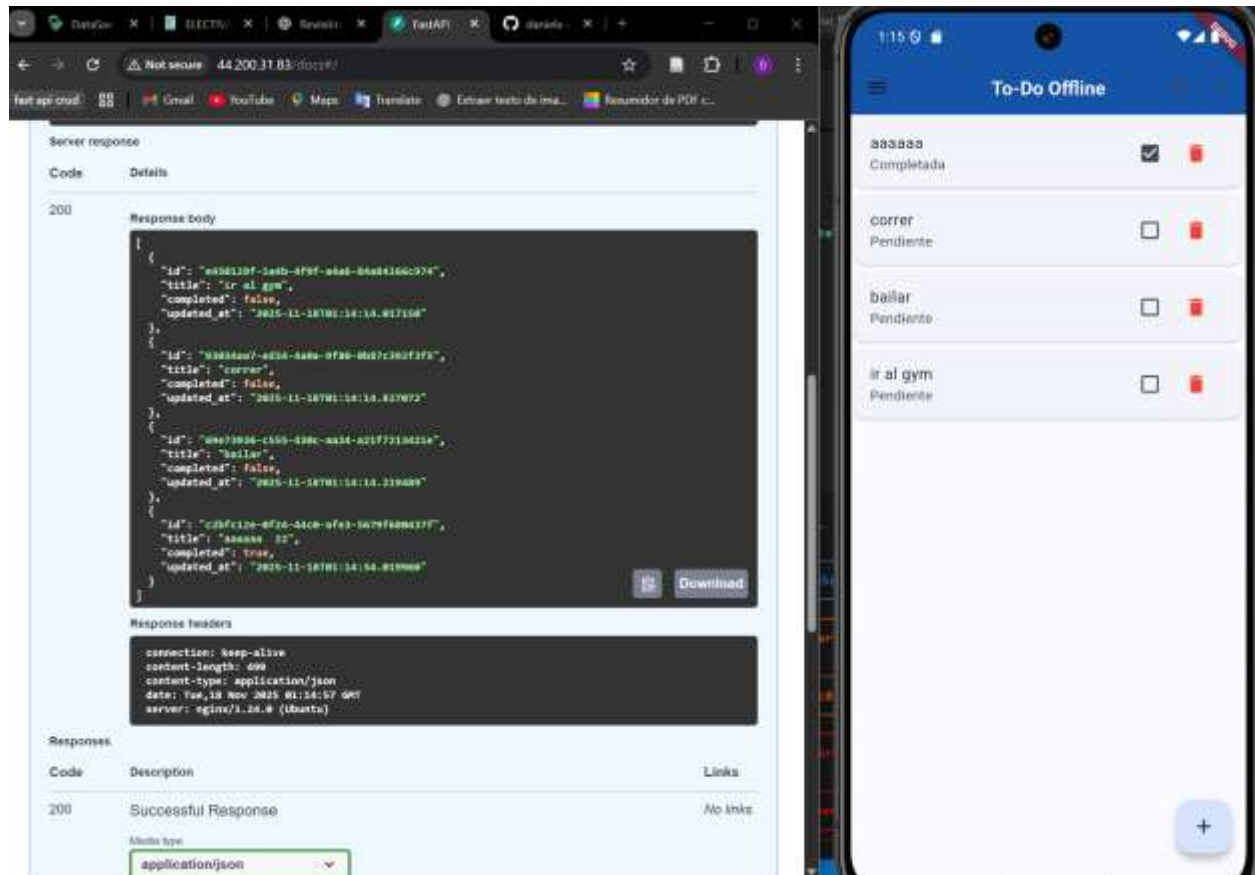
Code	Description	Links
200	Successful Response	No links

The mobile app interface on the right shows a list of tasks under the heading "To-Do Offline". The tasks are:

- ☒ aaaaaa (2025-11-18 00:03:59.186603)
- ☐ holaaaa (2025-11-18 00:02:55.191325)
- ☒ ir al gym (2025-11-17 23:46:57.915210)

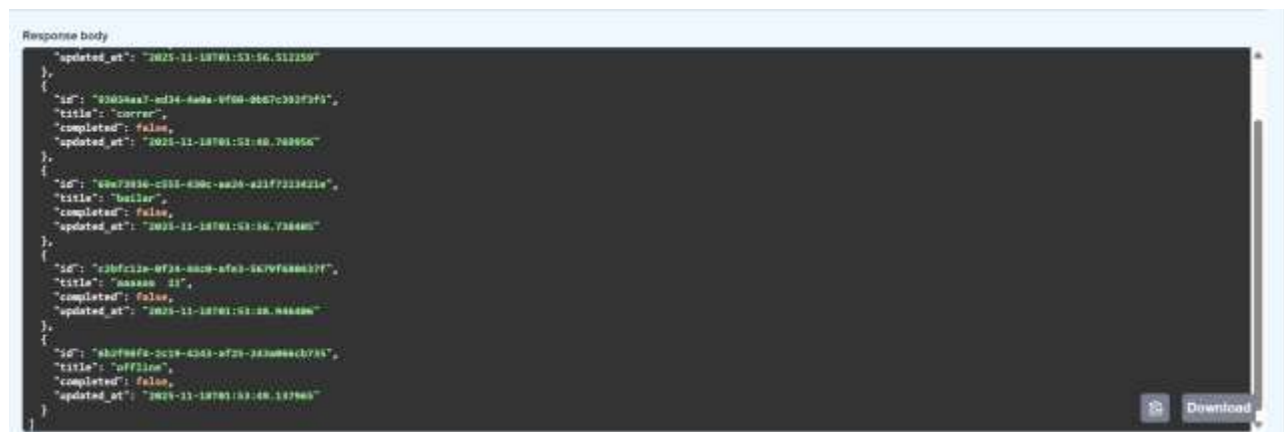


UI final:



Se hacen las pruebas respectivas desde un celular Android y funciona offline, una vez regresa el internet se sincroniza:

Api: http://44.200.31.83/docs#/default/get_all_tasks_tasks_get



Prueba desde API:

Raw Value : JSON

```
{
  "id": "0304aa7-ed34-4a6a-9f80-8887c80f319f",
  "title": "prueba desde api",
  "completed": false,
  "updated_at": "2025-11-18T02:02:48.406Z"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://64.296.31.83/tasks' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": "0304aa7-ed34-4a6a-9f80-8887c80f319f",
    "title": "prueba desde api",
    "completed": false,
    "updated_at": "2025-11-18T02:02:48.406Z"
  }'
```

Request URL

http://64.296.31.83/tasks

Server response

Code Details

200

Response body

```
{
  "id": "0304aa7-ed34-4a6a-9f80-8887c80f319f",
  "title": "prueba desde api",
  "completed": false,
  "updated_at": "2025-11-18T02:02:48.406Z"
}
```

Response headers

6 Generación de APK

1.1.0 (3)

17 de noviembre de 2025, 8:52:11 p.m. UTC-5

VERSION 1.1.0 - RELEASE NOTES! Novedades: Modo o...

[Copiar](#) [Descargar](#) [Borrar](#)

[Verificadores](#) [Notas de la versión](#) [Comentarios de verificadores](#) [Resultados de la prueba automatizada](#) BETA

 mariamaringarcia2000@gmail.com

✓

—

✓

—

✓

Descargado

 dduaran@uceva.edu.co

✓

—

✓

Aceptada

 Daniela Erazo marín
derazomarin10@gmail.com

✓

—

✓

Aceptada

 alejandro.ocampo01@uceva.edu.co

✓

Invitado

1.1.0 (3)

17 de noviembre de 2025, 8:52:11 p.m. UTC-5

VERSION 1.1.0 - RELEASE NOTES! Novedades: Modo o...

[Copiar](#) [Descargar](#) [Borrar](#)

[Verificadores](#) [Notas de la versión](#) [Comentarios de verificadores](#) [Resultados de la prueba automatizada](#) BETA

VERSION 1.1.0 - RELEASE NOTES!

Novedades:

Modo offline completo usando SQLite.

Las tareas se guardan localmente y se sincronizan con el servidor cuando vuelve la conexión.

Sincronización automática en segundo plano.

Botón Sync para sincronización manual.

Funciones principales:

Se pueden ver las notas de la actualización en los correos electrónicos de las actualizaciones nuevas y en la app para verificadores de App Distribution

VERSION 1.1.0 - RELEASE NOTES!

Novedades:

- Modo offline completo usando SQLite.
- Las tareas se guardan localmente y se sincronizan con el servidor cuando vuelve la conexión.
- Sincronización automática en segundo plano.
- Botón Sync para sincronización manual.

Funciones principales:

- Crear tareas.
- Editar tareas.
- Marcar tareas como completadas o pendientes.
- Eliminar tareas.
- Filtrar por todas, pendientes o completadas.
- Lista actualizada en tiempo real.

Arquitectura:

- Manejo de estado con Provider.
- Repositorio con datasources local (SQLite) y remoto (API AWS).
- Servicio de sincronización de operaciones en cola.

Mejoras incluidas:

- Corrección de duplicados al sincronizar.
- Mejor rendimiento al cargar y refrescar tareas.
- La UI ahora se actualiza correctamente después de eliminar o cambiar el estado de una tarea.
- Manejo optimizado del campo updatedAt.

Cómo probar:

- Para probar el modo offline:
- Desactiva la conexión.
- Crea o edita tareas
- Reactiva internet.
- Pulsa el botón Sync y verifica que los cambios llegan al servidor.

En modo online:

- Las tareas se sincronizan automáticamente y SQLite se mantiene actualizado.

Errores conocidos:

- En algunos dispositivos el check de completado puede tardar un momento en reflejarse mientras se recarga la lista desde SQLite.
- La sincronización puede ser lenta con conexiones inestables.

Próximas mejoras:

- Modo oscuro.
- Notificaciones.
- Subtareas y prioridades.
- Sistema de usuarios.